BookMyShow-lite Backend

Phase-wise Roadmap and Task Specification

Backend revision project for KC | October 2026

About this roadmap

Goal: build one production-style ticketing backend (browse movies, pick seats, hold them, pay, get a ticket) that revises every core backend skill expected from a developer with about 1.5 years of experience: API design, database design, authentication, concurrency, payments, background jobs, caching, testing and deployment.

Recommended stack: Node.js + Express (TypeScript recommended), PostgreSQL with Prisma or Sequelize, Redis + BullMQ, Razorpay (test mode), Docker + GitHub Actions, Jest/Vitest + Supertest, k6 for load tests.

How to use it

- Work phase by phase. Move on only when every 'Done When' check of the phase passes.
- Write the code yourself. Read docs to understand concepts; do not copy-paste solutions.
- Make it work first, then make it clean, then make it fast.
- After each phase: add a git tag, update the README and write 5 lines on what you learned.
- Write tests along with each phase instead of leaving everything for Phase 8.

Conventions used in the schemas

- PostgreSQL column types are used. Primary keys are UUID unless stated otherwise.
- All money is stored as INTEGER paise (never float). All timestamps are TIMESTAMPTZ in UTC.
- Soft delete is done with a nullable deleted_at column where it is mentioned.

Phases at a glance

No.	Phase	Goal	Tasks
0	**Planning, Setup & Foundation**	a clean skeleton that every other feature plugs into	7
1	**Authentication & Authorization (Login, Register, OAuth)**	a secure identity system, not just login/signup	10
2	**Catalog (Cities, Venues, Screens, Movies, Shows)**	all data that a customer browses before booking	5
3	**Seat Map & Show Inventory**	per-show seat availability that the booking engine will lock	3
4	**Booking Engine**	the heart of the project - no double booking, ever	6
5	**Payments & Refunds (Razorpay)**	money and booking state always stay consistent	5
6	**Async Jobs & Notifications**	slow work happens outside the request, and survives crashes	4
7	**Performance, Caching & Security**	make it fast and hard to abuse	4
8	**Testing**	refactor with confidence (write tests along with every phase, use this phase to fill gaps)	4
9	**DevOps, Observability & Deployment**	ship it like a real service	4
10	**Bonus (Senior-level push)**	optional features that make the project stand out	5

Phase 0: Planning, Setup & Foundation

Goal: a clean skeleton that every other feature plugs into

0.1 Project setup & folder structure

Description	Create a Git repo (main + feature branches, conventional commits). Setup Node.js (LTS) + Express, TypeScript recommended. Create a layered structure: routes > controllers > services > repositories, plus middlewares, utils, config and jobs folders. Add ESLint, Prettier, a pre-commit hook and .env.example.
API Endpoints	`GET /health`
Done When	**App starts with one command, /health returns 200 with uptime, lint passes.**

0.2 ER diagram & API contract

Description	Before coding, design the ER diagram for every entity (users, roles, venues, screens, seats, movies, shows, show_seats, bookings, payments, refunds...) using dbdiagram.io. Write the endpoint list with method, access level (public / customer / organizer / admin), request and response shape. Keep both inside a /docs folder.
Validation & Rules	Decide upfront and follow everywhere: money = integer paise (never float), time = UTC TIMESTAMPTZ, primary keys = UUID.
Done When	**ER diagram + endpoint list committed in /docs.**

0.3 Config & environment validation

Description	Create a config module that reads all env variables (DB URL, JWT secrets, Redis URL, Razorpay keys, SMTP, Google OAuth keys) and validates them at startup. If any required variable is missing or invalid, the app must crash immediately with a clear message.
Validation & Rules	Validate env with Zod/Joi. JWT secrets min 32 chars. PORT must be numeric. Never log secret values.
Done When	**Removing a required env var stops the app on boot.**

0.4 Request validation middleware

Description	Build a reusable validation middleware (Zod/Joi) that validates body, query and params. It should collect ALL errors and return them together in one response instead of failing on the first one.
Validation & Rules	Reject unknown fields (prevents mass assignment). Trim strings, lowercase emails. Error shape: [{field, message}] with status 400.
Done When	**Any route can be protected by passing a schema; invalid input never reaches the controller.**

0.5 Error handling & response format

Description	Create custom error classes (ValidationError 400, UnauthorizedError 401, ForbiddenError 403, NotFoundError 404, ConflictError 409, TooManyRequestsError 429) and ONE global error handler. Wrap async controllers so errors are never swallowed.

	Use one response format everywhere: success = {success, data, meta}, error = {success:false, error:{code, message, details}}.
Validation & Rules	Never send stack traces in production. Unknown errors return a generic 500 and are logged with the request-id.
Done When	**Every failure has the same shape; an unexpected throw gives 500 without leaking internals.**

0.6 Logging, request-id & Swagger docs

Description	Add pino structured logging with a request-id per request (accept x-request-id or generate one) and log method, path, status and latency. Serve OpenAPI/Swagger docs and keep them updated for every endpoint you build.
API Endpoints	`GET /docs (Swagger UI)`
Validation & Rules	Mask password, tokens, authorization header and payment data in logs.
Done When	**One request can be traced across logs by request-id; Swagger lists every built endpoint.**

0.7 Database, migrations & seeders

Description	Setup PostgreSQL with Prisma or Sequelize: connection pool, migration workflow, seed scripts (roles, a few cities, a demo admin). Implement graceful shutdown so the HTTP server and DB connections close cleanly on SIGTERM.
Validation & Rules	Migrations are forward-only and committed to Git. Seed scripts must be re-runnable (idempotent).
Database	**roles** - id SMALLSERIAL PK - name VARCHAR(30) NOT NULL UNIQUE (customer / organizer / admin)
Done When	**Fresh clone: migrate + seed gives a working DB; Ctrl+C shuts down without hanging connections.**

Phase 1: Authentication & Authorization (Login, Register, OAuth)

Goal: a secure identity system, not just login/signup

1.1 Register API

Description	Create a register API where the user shares: full name, the username he wants, email, password and confirm password. After validation, hash the password (argon2/bcrypt), create the user with role = customer and email_verified = false, and generate an email verification token (email is sent through the queue in Phase 6; until then just log the link). Return 201 with the user (never the password hash).
API Endpoints	`POST /api/v1/auth/register` `Body: {full_name, username, email, password, confirm_password}`
Validation & Rules	• full_name: 2-100 chars, letters/spaces/dots only. • username: 3-20 chars, lowercase letters, numbers, underscore, dot; must start with a letter; no consecutive dots/underscores; reserved words blocked (admin, support, root, api, system). • email: valid format, lowercased, max 254 chars. • password: 8-64 chars, at least 1 upper, 1 lower, 1 number, 1 special char; must not contain username or email. • confirm_password must match password. • Username/email already used -> 409 (also keep DB UNIQUE constraints to handle race conditions).
Database	**users** - id UUID PK DEFAULT gen_random_uuid() - full_name VARCHAR(100) NOT NULL - username VARCHAR(20) NOT NULL UNIQUE (store lowercase) - email VARCHAR(254) NOT NULL UNIQUE (store lowercase) - password_hash VARCHAR(255) NULL (NULL for OAuth-only users) - role_id SMALLINT NOT NULL FK roles(id) DEFAULT customer - email_verified BOOLEAN NOT NULL DEFAULT false - is_active BOOLEAN NOT NULL DEFAULT true - failed_login_attempts INT NOT NULL DEFAULT 0 - locked_until TIMESTAMPTZ NULL - last_login_at TIMESTAMPTZ NULL - created_at TIMESTAMPTZ NOT NULL DEFAULT now() - updated_at TIMESTAMPTZ NOT NULL DEFAULT now() - deleted_at TIMESTAMPTZ NULL (soft delete) Indexes: UNIQUE(username), UNIQUE(email)
Done When	**Valid registration creates a user; every invalid case returns clear field-wise errors; password is never stored or logged in plain text.**

1.2 Username availability check + suggestions

Description	Create an API to check whether a username is already taken by another user. If it is taken, return 2-3 random available suggestions built from the requested name (for example name + random 2-3 digits, name + underscore + word, name + dot + year). Every suggestion must be re-checked against the DB before returning. The frontend will call this (debounced) while the user types in the register form.

API Endpoints	`GET /api/v1/auth/username-availability?username=kc_dev`
	`Response: {available: true}`
	`or {available: false, suggestions: ['kc_dev42', 'kc.dev', 'kc_dev_07']}`

Validation & Rules	• Same format rules as register username; invalid format -> 400 with the reason (no suggestions). • Case-insensitive check.

	• Rate limit (e.g. 10 requests/min per IP) so nobody can scrape existing usernames. • Suggestions must be unique, valid and not reserved words.
Database	`No new table. Uses users.username (UNIQUE index).`
Done When	**Taken username returns 3 valid suggestions that are really free; free username returns available = true.**

1.3 Login API

Description	Create a login API where the user sends username OR email in one field plus password. Detect whether the identifier is an email (contains @ and valid) or a username, find the user case-insensitively, compare the password hash and return an access token + refresh token. Use the same generic message 'Invalid credentials' for wrong user and wrong password so attackers cannot find which accounts exist. Lock the account for 15 minutes after 5 failed attempts.
API Endpoints	`POST /api/v1/auth/login` `Body: {identifier, password}`
Validation & Rules	• identifier: required, trimmed, 3-254 chars. • password: required, max 64 chars. • Wrong user OR wrong password -> same 401 'Invalid credentials'. • 5 failed attempts -> locked_until = now + 15 min (423/429 with retry info). • Unverified email -> 403 with code EMAIL_NOT_VERIFIED. • Inactive/soft-deleted user cannot login. • On success: reset failed_login_attempts, update last_login_at.
Database	`Uses users (failed_login_attempts, locked_until, last_login_at). Tokens are stored in refresh_tokens (task 1.4).`
Done When	**Login works with both username and email; response time is similar for existing and non-existing users.**

1.4 Access & refresh tokens with rotation

Description	Issue a short-lived access JWT (about 15 min) containing user id and role, and a long-lived refresh token (7-30 days) as a random opaque string. Store only the SHA-256 hash of the refresh token in DB along with device info. On refresh, ROTATE: mark the old token revoked and issue a new one in the same token family. If a revoked token is used again, revoke the whole family (token theft detection). Send the refresh token in an httpOnly, Secure, SameSite cookie.
API Endpoints	`POST /api/v1/auth/refresh`
Validation & Rules	• Missing/expired/revoked refresh token -> 401. • Reuse of an already-rotated token -> revoke entire family + 401. • Access token verified with signature + expiry; never accept 'alg: none'. • Cookie flags: httpOnly, Secure, SameSite=Lax/Strict.

Database	**`refresh_tokens`**
	`- id UUID PK`
	`- user_id UUID NOT NULL FK users(id) ON DELETE CASCADE`
	`- token_hash CHAR(64) NOT NULL UNIQUE`
	`- family_id UUID NOT NULL`
	`- user_agent VARCHAR(255) NULL`
	`- ip_address INET NULL`
	`- expires_at TIMESTAMPTZ NOT NULL`
	`- revoked_at TIMESTAMPTZ NULL`
	`- replaced_by UUID NULL FK refresh_tokens(id)`

	- created_at TIMESTAMPTZ NOT NULL DEFAULT now() Indexes: (user_id), (family_id)
Done When	**Old refresh token stops working after one use; replaying it kills all sessions in that family.**

1.5 Logout & logout from all devices

Description	Logout revokes the current refresh token and clears the cookie. Logout-all revokes every active refresh token of the user (useful after password change or suspicious activity).
API Endpoints	POST /api/v1/auth/logout POST /api/v1/auth/logout-all
Validation & Rules	Both need a valid access token. Logout must be idempotent (calling twice does not error).
Database	Uses refresh_tokens (revoked_at).
Done When	**After logout the refresh endpoint rejects the old token.**

1.6 OAuth login (Google)

Description	Implement 'Login with Google' using OAuth 2.0 authorization code flow with state (and PKCE if possible). Backend redirects to Google, receives the callback, verifies state, exchanges the code for tokens and reads verified email + name. If an account with that email already exists, link Google to it; otherwise create a new user (email_verified = true, password_hash = NULL, auto-generated unique username using the suggestion logic from 1.2). Finally issue the same access + refresh tokens as normal login.
API Endpoints	GET /api/v1/auth/oauth/google (redirect) GET /api/v1/auth/oauth/google/callback
Validation & Rules	• Validate state parameter (CSRF protection). Accept only if Google says email_verified = true. • Link to an existing local account only if that account's email is verified; otherwise reset its password hash and mark verified (prevents pre-account takeover). • Handle user cancelling consent (error=access_denied) gracefully. • One Google account can link to only one user.
Database	**oauth_accounts** - id UUID PK - user_id UUID NOT NULL FK users(id) ON DELETE CASCADE - provider VARCHAR(20) NOT NULL (google) - provider_user_id VARCHAR(100) NOT NULL - provider_email VARCHAR(254) NULL - created_at TIMESTAMPTZ NOT NULL DEFAULT now() Constraint: UNIQUE(provider, provider_user_id)
Done When	**New Google user is created with unique username; existing user gets linked; same tokens as normal login.**

1.7 Email verification

Description	After registration the user gets a one-time link/token. Verifying it sets email_verified = true. Provide a resend API. Tokens are random, stored hashed, single-use and expiring.
API Endpoints	POST /api/v1/auth/verify-email Body: {token} POST /api/v1/auth/resend-verification Body: {email}

Validation & Rules	• Token expiry 24 hours, single use (used_at set). • Resend limited to 3 per hour per email. • Resend always returns the same message (no account enumeration). • Already verified -> 200 no-op.
Database	**one_time_tokens** - id UUID PK - user_id UUID NOT NULL FK users(id) ON DELETE CASCADE - type VARCHAR(20) NOT NULL (EMAIL_VERIFY / PASSWORD_RESET) - token_hash CHAR(64) NOT NULL UNIQUE - expires_at TIMESTAMPTZ NOT NULL - used_at TIMESTAMPTZ NULL - created_at TIMESTAMPTZ NOT NULL DEFAULT now() Index: (user_id, type)
Done When	**Valid token verifies the user once; expired/used token is rejected.**

1.8 Forgot & reset password

Description	Forgot-password sends a reset link to the email if the account exists, but ALWAYS returns the same response. Reset-password accepts token + new password + confirm password, updates the hash and revokes all refresh tokens of that user.
API Endpoints	POST /api/v1/auth/forgot-password Body: {email} POST /api/v1/auth/reset-password Body: {token, new_password, confirm_password}
Validation & Rules	• Reset token expiry 30 min, single use. • New password follows the same strength rules and must differ from the old one. • confirm_password must match. • After reset: all sessions logged out, failed_login_attempts reset.
Database	Uses one_time_tokens (type = PASSWORD_RESET) and refresh_tokens.
Done When	**Cannot learn which emails exist; old sessions die after reset.**

1.9 Profile (me) & change password

Description	Logged-in user can view and update own profile and change password. Changing username uses the same availability check; changing email requires re-verification.
API Endpoints	GET /api/v1/users/me PATCH /api/v1/users/me Body: {full_name?, username?} POST /api/v1/users/me/change-password Body: {current_password, new_password, confirm_password}
Validation & Rules	• Change password needs correct current_password; new must differ and follow strength rules. • Username change: must be available; limit to once per 30 days. • Whitelist updatable fields (user cannot update role_id or email_verified).
Database	Add to users: username_changed_at TIMESTAMPTZ NULL
Done When	**User cannot escalate role or touch other fields through PATCH.**

1.10 RBAC & ownership middleware

Description	Build 'authenticate' middleware (verifies access token, attaches user) and 'authorize(...roles)' middleware (customer / organizer / admin). Ownership checks go in

	the service layer: a customer sees only own bookings, an organizer edits only own venues.
API Endpoints	`Applies to all protected routes`
Validation & Rules	• No token -> 401. Wrong role -> 403. • Resource of another user -> return 404 (do not reveal that it exists). • Never trust role/user id coming from request body.
Done When	**An organizer cannot edit another organizer's venue; a customer cannot read another customer's booking.**

Phase 2: Catalog (Cities, Venues, Screens, Movies, Shows)

Goal: all data that a customer browses before booking

2.1 Cities & venues

Description	Admin manages cities. An organizer registers a venue (name, address, city, contact). A venue stays hidden until admin approves it.

API Endpoints	`GET /api/v1/cities`
	`POST/PATCH /api/v1/admin/cities`
	`POST /api/v1/organizer/venues`
	`PATCH /api/v1/organizer/venues/:id`
	`PATCH /api/v1/admin/venues/:id/approve (or /reject)`

Validation & Rules	• City: name + state unique. Venue name 3-100 chars, address required, phone = valid 10-digit Indian number. • Status flow: PENDING -> APPROVED / REJECTED. • Only the owner organizer can edit own venue. • Public APIs return only APPROVED venues.
Database	**cities** - id SERIAL PK - name VARCHAR(80) NOT NULL - state VARCHAR(80) NOT NULL - is_active BOOLEAN NOT NULL DEFAULT true - UNIQUE(name, state) **venues** - id UUID PK - owner_id UUID NOT NULL FK users(id) - city_id INT NOT NULL FK cities(id) - name VARCHAR(100) NOT NULL - address TEXT NOT NULL - latitude NUMERIC(9,6) NULL - longitude NUMERIC(9,6) NULL - contact_phone VARCHAR(15) NULL - status VARCHAR(10) NOT NULL DEFAULT 'PENDING' CHECK IN (PENDING, APPROVED, REJECTED) - created_at / updated_at TIMESTAMPTZ, deleted_at TIMESTAMPTZ NULL Index: (city_id, status)
Done When	**Unapproved venue is invisible to public; only owner can modify it.**

2.2 Screens & seat layout

Description	A venue has multiple screens. Organizer defines the layout: rows (A, B, C...), seats per row and seat type (REGULAR, PREMIUM, RECLINER). Generate all seat rows in one DB transaction. Once any show exists on that screen, layout becomes locked.

API Endpoints	`POST /api/v1/organizer/venues/:venueId/screens`
	`GET /api/v1/organizer/screens/:id`
	`PUT /api/v1/organizer/screens/:id/layout`

Validation & Rules	• Max 26 rows, max 50 seats per row. Seat label (A1, A2...) unique within a screen. Screen name unique within venue. • Layout cannot change if shows exist (409). • Only venue owner can create/edit screens.
Database	**screens** - id UUID PK - venue_id UUID NOT NULL FK venues(id)

	- name VARCHAR(50) NOT NULL - total_seats INT NOT NULL - created_at TIMESTAMPTZ NOT NULL DEFAULT now() - UNIQUE(venue_id, name) **seats** - id UUID PK - screen_id UUID NOT NULL FK screens(id) ON DELETE CASCADE - row_label VARCHAR(3) NOT NULL - seat_number SMALLINT NOT NULL - seat_type VARCHAR(10) NOT NULL DEFAULT 'REGULAR' - UNIQUE(screen_id, row_label, seat_number)
Done When	**A 10x12 screen creates 120 seat rows in one transaction; failing midway creates nothing.**

2.3 Movies / events

Description	Admin (or organizer) creates movies with title, description, duration, language, genres, certificate, release date and poster URL. Public listing supports filters, search and pagination.
API Endpoints	POST /api/v1/admin/movies PATCH /api/v1/admin/movies/:id GET /api/v1/movies?city=&genre=&language=&q=&page=&limit=&sort= GET /api/v1/movies/:id
Validation & Rules	• title 1-150 chars; duration_minutes 30-360; certificate IN (U, UA, A, S); release_date valid date; poster_url valid https URL. • page >= 1, limit 1-50 (default 10); sort only from a whitelist (release_date, title) to avoid injection.
Database	**movies** - id UUID PK - title VARCHAR(150) NOT NULL - description TEXT NULL - duration_minutes SMALLINT NOT NULL - language VARCHAR(30) NOT NULL - genres TEXT[] NOT NULL - certificate VARCHAR(3) NULL - release_date DATE NULL - poster_url VARCHAR(500) NULL - status VARCHAR(12) NOT NULL DEFAULT 'UPCOMING' CHECK IN (UPCOMING, NOW_SHOWING, ENDED) - created_by UUID FK users(id) - created_at / updated_at TIMESTAMPTZ, deleted_at TIMESTAMPTZ NULL Indexes: GIN(genres), (status, release_date)
Done When	**Listing supports filter + search + pagination with total count in meta.**

2.4 Shows & pricing

Description	Organizer schedules a show: movie + screen + start time + price for each seat type. End time = start + movie duration + cleaning buffer (e.g. 15 min). Two shows on the same screen must never overlap.
API Endpoints	POST /api/v1/organizer/shows GET /api/v1/shows?movieId=&cityId=&date= PATCH /api/v1/organizer/shows/:id POST /api/v1/organizer/shows/:id/cancel

Validation & Rules	• start_time must be in the future. • No overlap of [start_time, end_time] on the same screen (check in service AND enforce in DB). • Price > 0, stored as integer paise. • Only the owner of the screen's venue can create/cancel. • Store UTC; convert to IST only in the response layer.
Database	**shows** - id UUID PK - movie_id UUID NOT NULL FK movies(id) - screen_id UUID NOT NULL FK screens(id) - start_time TIMESTAMPTZ NOT NULL - end_time TIMESTAMPTZ NOT NULL - status VARCHAR(10) NOT NULL DEFAULT 'SCHEDULED' CHECK IN (SCHEDULED, CANCELLED, COMPLETED) - created_at / updated_at TIMESTAMPTZ - Optional DB rule: EXCLUDE USING gist (screen_id WITH =, tstzrange(start_time, end_time) WITH &&) Indexes: (screen_id, start_time), (movie_id, start_time) **show_prices** - id UUID PK - show_id UUID NOT NULL FK shows(id) ON DELETE CASCADE - seat_type VARCHAR(10) NOT NULL - price_paise INT NOT NULL CHECK (price_paise > 0) - UNIQUE(show_id, seat_type)
Done When	**Overlapping show creation is rejected even when two requests arrive at the same time.**

2.5 Query optimization on listings

Description	Run EXPLAIN ANALYZE on the movie listing and show listing queries, add composite indexes, remove N+1 queries (eager load), and convert one list to cursor pagination. Write before/after timings in /docs.
Validation & Rules	Do not add indexes blindly - prove each one with EXPLAIN output.
Done When	**At least two queries tuned with documented numbers.**

Phase 3: Seat Map & Show Inventory

Goal: per-show seat availability that the booking engine will lock

3.1 Generate show inventory

Description	When a show is created, generate one show_seats row for every seat of the screen in the SAME transaction (bulk insert), copying the seat type and the price from show_prices. This table is where all concurrency happens, so design it carefully.
Validation & Rules	Show creation + inventory creation is all-or-nothing. A show must never exist without its inventory.
Database	**show_seats** - id UUID PK - show_id UUID NOT NULL FK shows(id) ON DELETE CASCADE - seat_id UUID NOT NULL FK seats(id) - status VARCHAR(10) NOT NULL DEFAULT 'AVAILABLE' CHECK IN (AVAILABLE, HELD, BOOKED) - price_paise INT NOT NULL - held_by UUID NULL FK users(id) - held_until TIMESTAMPTZ NULL - booking_id UUID NULL FK bookings(id) - version INT NOT NULL DEFAULT 0 - UNIQUE(show_id, seat_id) Index: (show_id, status)
Done When	**A show on a 200-seat screen creates 200 inventory rows in one transaction.**

3.2 Seat map API

Description	Return the seat grid of a show grouped by row, with seat label, type, price and status (AVAILABLE / HELD / BOOKED). A HELD seat whose held_until has passed must be shown as AVAILABLE. Fetch everything in one query (no N+1).
API Endpoints	`GET /api/v1/shows/:id/seats`
Validation & Rules	• Show must exist, be SCHEDULED and in the future, else 404. • Never expose who holds a seat (held_by) to other users.
Database	`Reads show_seats + seats.`
Done When	**Seat map loads with one or two queries regardless of seat count.**

3.3 Show cancellation handling

Description	When an organizer cancels a show that already has confirmed bookings: mark the show CANCELLED, mark every booking for refund (refund flow comes in Phase 5) and queue notification emails (Phase 6).
API Endpoints	`POST /api/v1/organizer/shows/:id/cancel (from 2.4)`
Validation & Rules	Only SCHEDULED shows can be cancelled. Whole operation in a transaction.
Database	`Uses shows.status, bookings.status.`
Done When	**Cancelling a show with 10 bookings flags all 10 for refund.**

Phase 4: Booking Engine

Goal: the heart of the project - no double booking, ever

4.1 Booking schema & state machine

Description	Define the booking lifecycle: PENDING -> CONFIRMED / EXPIRED / CANCELLED, and CONFIRMED -> CANCELLED. Reject every other transition in one central function so the rule lives in one place.
Validation & Rules	Invalid transition -> 409. total_amount is always calculated on the server from DB prices - never trust the amount from the client.
Database	**bookings** - id UUID PK - booking_ref VARCHAR(12) NOT NULL UNIQUE (human readable code) - user_id UUID NOT NULL FK users(id) - show_id UUID NOT NULL FK shows(id) - status VARCHAR(10) NOT NULL DEFAULT 'PENDING' CHECK IN (PENDING, CONFIRMED, EXPIRED, CANCELLED) - total_amount_paise INT NOT NULL - currency CHAR(3) NOT NULL DEFAULT 'INR' - idempotency_key VARCHAR(64) NULL - expires_at TIMESTAMPTZ NOT NULL - confirmed_at / cancelled_at TIMESTAMPTZ NULL - ticket_url VARCHAR(500) NULL - created_at / updated_at TIMESTAMPTZ - UNIQUE(user_id, idempotency_key) Indexes: (user_id, created_at DESC), (show_id, status) **booking_items** - id UUID PK - booking_id UUID NOT NULL FK bookings(id) ON DELETE CASCADE - show_seat_id UUID NOT NULL FK show_seats(id) - price_paise INT NOT NULL
Done When	**Transition function has tests for every allowed and forbidden move.**

4.2 Hold seats API (DB locking)

Description	User selects seats and the API holds them for 8 minutes. In ONE transaction lock the selected show_seats in a fixed order (SELECT ... FOR UPDATE ordered by id) or use an atomic conditional UPDATE (set HELD only WHERE status = AVAILABLE or hold expired). Create a PENDING booking + items and calculate the total. All-or-nothing: if even one seat is unavailable, hold none.
API Endpoints	POST /api/v1/bookings Header: Idempotency-Key Body: {showId, seatIds: []}
Validation & Rules	• 1 to 6 seats per booking; no duplicate seat ids; all seats must belong to the given show. • Show must be SCHEDULED and in the future. • Unavailable seats -> 409 with the list of unavailable seat ids. • Same Idempotency-Key = same booking returned (no duplicate). • Lock seats in sorted order to avoid deadlocks.
Database	Uses show_seats (status, held_by, held_until, booking_id), bookings, booking_items.
Done When	**50 parallel requests for the same seat -> exactly 1 success.**

4.3 Redis lock approach + comparison

Description	Implement a second approach: Redis SET NX PX lock per seat (key like seat:{showId}:{seatId}) before the DB write. Compare both approaches on correctness, latency and failure cases (what if Redis dies?). Write a short Architecture Decision Record explaining which approach you keep and why.
Validation & Rules	Always set a TTL on locks. Release only a lock you own (compare lock value).
Done When	**ADR committed in /docs with measured numbers.**

4.4 Hold expiry & release

Description	A scheduled job every minute moves expired PENDING bookings to EXPIRED and sets their seats back to AVAILABLE. Also do a lazy check inside the seat-map and hold APIs so correctness does not depend only on the job running.
Validation & Rules	Release must be idempotent and run in a transaction. Never release seats of a CONFIRMED booking.
Database	`Uses show_seats, bookings.expires_at.`
Done When	**No seat stays HELD after its hold time even if the cron job is stopped.**

4.5 Get, list & cancel booking

Description	Customer can fetch one booking, list own bookings with filters, and cancel a CONFIRMED booking. Cancel releases seats and starts the refund based on the cancellation policy (Phase 5).

API Endpoints	`GET /api/v1/bookings/:id`
	`GET /api/v1/bookings?status=&page=&limit=`
	`POST /api/v1/bookings/:id/cancel`

Validation & Rules	• Customer sees only own bookings (admin can see all). • Cancel allowed only for CONFIRMED bookings and up to a cutoff (e.g. 2 hours before show).
Database	`Uses bookings, booking_items, show_seats.`
Done When	**Another user's booking id returns 404.**

4.6 Concurrency test script

Description	Write a script that fires 50 parallel hold requests for the SAME seat and also for overlapping seat sets (e.g. [A1,A2] vs [A2,A3]). Assert: exactly one winner, no deadlock, no seat stuck in HELD.
Done When	**Script passes 20 times in a row.**

Phase 5: Payments & Refunds (Razorpay)

Goal: money and booking state always stay consistent

5.1 Create payment order

Description	For an own PENDING and non-expired booking, create a Razorpay order for the booking total (in paise) and return order id + public key to the client. One active payment per booking - if an order already exists, return it again.
API Endpoints	`POST /api/v1/bookings/:id/payments`
Validation & Rules	• Booking must be PENDING, owned by user and not expired. • Amount always from DB (integer paise). • Idempotent: repeated call returns the same order.
Database	**payments** - id UUID PK - booking_id UUID NOT NULL FK bookings(id) - provider VARCHAR(20) NOT NULL DEFAULT 'RAZORPAY' - provider_order_id VARCHAR(40) NOT NULL UNIQUE - provider_payment_id VARCHAR(40) NULL UNIQUE - amount_paise INT NOT NULL - currency CHAR(3) NOT NULL DEFAULT 'INR' - status VARCHAR(18) NOT NULL DEFAULT 'CREATED' CHECK IN (CREATED, AUTHORIZED, CAPTURED, FAILED, REFUNDED, PARTIALLY_REFUNDED) - method VARCHAR(20) NULL - failure_reason VARCHAR(255) NULL - created_at / updated_at TIMESTAMPTZ **payment_events** - id UUID PK - payment_id UUID NULL FK payments(id) - provider_event_id VARCHAR(60) NOT NULL UNIQUE - event_type VARCHAR(40) NOT NULL - payload JSONB NOT NULL - processed_at TIMESTAMPTZ NULL - created_at TIMESTAMPTZ NOT NULL DEFAULT now()
Done When	**Order is created once per booking and amount matches booking total.**

5.2 Client payment verification

Description	After checkout the client sends order id, payment id and signature. Verify the HMAC signature on the server. This only gives quick feedback to the user; the webhook stays the source of truth for confirming the booking.
API Endpoints	`POST /api/v1/payments/verify` `Body: {razorpay_order_id, razorpay_payment_id, razorpay_signature}`
Validation & Rules	• Signature mismatch -> 400. • Order must belong to the logged-in user's booking. • Use constant-time comparison.
Database	`Uses payments.`
Done When	**Tampered signature is rejected.**

5.3 Razorpay webhook handler

Description	Receive Razorpay webhooks on a raw-body endpoint and verify X-Razorpay-Signature. Save each event in payment_events (unique event id = dedupe). On payment.captured, in one transaction: payment = CAPTURED, booking = CONFIRMED, seats = BOOKED. Respond 200 quickly and push slow work (email, PDF) to the queue.
API Endpoints	`POST /api/v1/webhooks/razorpay`
Validation & Rules	• Invalid signature -> 400 and log it. • Duplicate event id -> 200 and do nothing. • Unknown order -> log + 200. • Out-of-order events must not move a payment backwards (e.g. FAILED after CAPTURED).
Database	`Uses payments, payment_events, bookings, show_seats.`
Done When	**Replaying the same webhook 3 times confirms the booking only once.**

5.4 Late / failed payment handling

Description	If payment is captured after the booking already expired or the seats were taken, mark booking EXPIRED and start an automatic refund. On payment.failed release the seats immediately (or allow retry while the hold is still active).
Validation & Rules	Never confirm a booking whose seats are no longer held by that booking.
Database	`Uses payments, bookings, refunds.`
Done When	**Late payment scenario ends with money refunded and no double-allocated seat.**

5.5 Refunds & cancellation policy

Description	Implement a simple policy (example: more than 24h before show = 100%, 2-24h = 50%, under 2h = no refund). Call the Razorpay refund API and update status through refund webhooks.
API Endpoints	`POST /api/v1/bookings/:id/cancel (from 4.5)` `GET /api/v1/bookings/:id/refund`
Validation & Rules	• Refund amount <= captured amount, in integer paise. • Only one refund per booking cancellation (idempotent). • Show-cancelled-by-organizer = always 100%.
Database	**refunds** - id UUID PK - payment_id UUID NOT NULL FK payments(id) - booking_id UUID NOT NULL FK bookings(id) - provider_refund_id VARCHAR(40) NULL UNIQUE - amount_paise INT NOT NULL CHECK (amount_paise > 0) - status VARCHAR(10) NOT NULL DEFAULT 'INITIATED' CHECK IN (INITIATED, PROCESSED, FAILED) - reason VARCHAR(100) NULL - created_at / updated_at TIMESTAMPTZ
Done When	**Refund amounts follow the policy table exactly; retries do not refund twice.**

Phase 6: Async Jobs & Notifications

Goal: slow work happens outside the request, and survives crashes

6.1 Queue & worker setup (BullMQ)

Description	Run a separate worker process with Redis + BullMQ. Create queues: emails, tickets, maintenance, reminders. Retry failed jobs 5 times with exponential backoff, then move to a dead-letter queue. Add a queue dashboard protected for admin only.
API Endpoints	`GET /admin/queues (dashboard, admin only)`
Validation & Rules	Jobs must be idempotent (safe to run twice). Pass ids in job data, not whole objects.
Done When	**Killing the worker mid-job does not lose the job.**

6.2 Email notifications

Description	Send emails through the queue: verification, password reset, booking confirmation, cancellation/refund, show reminder. Use templates and a provider (Nodemailer/SMTP, Resend or Brevo). Log every attempt.
Validation & Rules	Never block API response on email sending. Retry on provider error, stop after max attempts and mark FAILED.
Database	**notification_logs** - id UUID PK - user_id UUID NULL FK users(id) - type VARCHAR(30) NOT NULL - channel VARCHAR(10) NOT NULL DEFAULT 'EMAIL' - recipient VARCHAR(254) NOT NULL - status VARCHAR(10) NOT NULL DEFAULT 'QUEUED' CHECK IN (QUEUED, SENT, FAILED) - attempts SMALLINT NOT NULL DEFAULT 0 - error TEXT NULL - created_at TIMESTAMPTZ NOT NULL DEFAULT now() - sent_at TIMESTAMPTZ NULL
Done When	**Booking confirmation email arrives after payment; API stays fast.**

6.3 Ticket PDF & QR code

Description	After booking confirmation, generate a QR code (signed booking_ref payload) and a PDF ticket, save it to file storage (local / S3-compatible) and attach it to the confirmation email. Provide an API to download it.
API Endpoints	`GET /api/v1/bookings/:id/ticket`
Validation & Rules	Only CONFIRMED and own bookings. QR payload must be signed so it cannot be forged.
Database	`Uses bookings.ticket_url (added in 4.1).`
Done When	**Downloaded ticket shows movie, show time, seats and a scannable QR.**

6.4 Scheduled jobs

Description	Add scheduled/delayed jobs: show reminder 2 hours before start, hold-expiry sweeper (from 4.4), nightly cleanup of expired one_time_tokens and refresh_tokens, and a daily admin summary report.

Validation & Rules	Scheduled jobs must not run twice when multiple workers are running (use repeatable jobs / unique job ids).
Database	`Uses one_time_tokens, refresh_tokens, bookings.`
Done When	**Reminder arrives about 2 hours before the show.**

Phase 7: Performance, Caching & Security

Goal: make it fast and hard to abuse

7.1 Redis caching

Description	Use cache-aside for cities, movie list and show list (per city/date) with a TTL of 60-300 seconds. Invalidate cache when admin/organizer updates data. Add protection against cache stampede (lock or TTL jitter). Do NOT cache seat availability for long.
Validation & Rules	Cache keys must include all filters. Seat map TTL max a few seconds, or skip cache.
Done When	**Second request to a listing is served from cache; update clears it.**

7.2 Rate limiting

Description	Add a global limiter and stricter limiters on login, register, username-availability, forgot-password and booking APIs. Use Redis-backed sliding window per IP and per user.
Validation & Rules	Return 429 with Retry-After header. Different limits per route group.
Done When	**Abusive client gets 429 and normal users are unaffected.**

7.3 Security hardening

Description	Add helmet, strict CORS whitelist, body-size limits, input sanitization, parameterized queries only, secure cookies and secrets from env. Run npm audit and go through the OWASP API Top 10 checklist (BOLA/IDOR, mass assignment, excessive data exposure, broken auth).
Validation & Rules	Response objects must expose only whitelisted fields (never password_hash, token hashes).
Done When	**Written checklist in /docs with each OWASP item marked and explained.**

7.4 Load test & DB tuning

Description	Load test the important endpoints (listing, seat map, hold seats) with k6. Check slow queries, tune indexes and connection pool, and record p95 latency before and after caching.
Done When	**Before/after p95 numbers documented in README.**

Phase 8: Testing

Goal: refactor with confidence (write tests along with every phase, use this phase to fill gaps)

8.1 Unit tests (services)

Description	Test business logic in isolation: password rules, username suggestions, booking state machine, refund policy calculation, price calculation.
Validation & Rules	Cover happy path + every validation error.
Done When	**Services layer coverage is at least 70%.**

8.2 Integration tests (API + real DB)

Description	Use Supertest with a real test PostgreSQL database (docker-compose). Reset DB between tests using transactions/truncate. Use factories for test data. Cover register, login, refresh rotation, catalog CRUD, booking flow.
Validation & Rules	Tests must not depend on execution order.
Done When	**Full register > login > book flow runs green.**

8.3 Concurrency & webhook tests

Description	Automate the parallel booking test from 4.6 and test webhooks with mocked Razorpay payloads (valid, invalid signature, duplicate, out-of-order, late payment).
Done When	**All edge cases from the list pass reliably (not flaky).**

8.4 Test automation in CI

Description	Run lint + tests on every push using GitHub Actions with Postgres and Redis service containers. Fail the build on test failure.
Done When	**Clean clone + one command runs all tests green.**

Phase 9: DevOps, Observability & Deployment

Goal: ship it like a real service

9.1 Dockerize

Description	Write a multi-stage Dockerfile (small image, non-root user) and docker-compose with api, worker, postgres and redis. Add health checks and volumes for DB data.
Validation & Rules	No secrets inside image or Git. Use .dockerignore.
Done When	**docker compose up starts the complete system.**

9.2 CI/CD pipeline

Description	GitHub Actions: lint > test > build image > push image > deploy. Run DB migrations as a controlled deploy step.
Validation & Rules	Deploy only from main after all checks pass.
Done When	**Push to main updates the live app automatically.**

9.3 Deployment

Description	Deploy on Render/Railway or a VPS with Nginx reverse proxy and HTTPS. Use separate dev/staging/prod environment variables and set up DB backups.
API Endpoints	`GET /health and GET /ready`
Validation & Rules	Health = process alive; readiness = DB + Redis reachable.
Done When	**Live URL with Swagger docs is working over HTTPS.**

9.4 Observability

Description	Expose basic metrics (request count, latency, error rate, queue length) with prom-client and optionally a Grafana dashboard. Make sure logs carry request-id and are readable in production.
API Endpoints	`GET /metrics (protected)`
Validation & Rules	Do not expose /metrics publicly.
Done When	**You can answer 'which endpoint is slowest right now?' from data.**

Phase 10: Bonus (Senior-level push)

Goal: optional features that make the project stand out

10.1 Live seat map (WebSocket / SSE)

Description	Push seat status changes (held / booked / released) to everyone viewing the same show in real time.
API Endpoints	`WS /ws/shows/:id  (or GET /api/v1/shows/:id/seats/stream for SSE)`
Validation & Rules	Authenticate the connection; limit connections per user.
Done When	**Two browsers see each other's holds instantly.**

10.2 Promo codes

Description	Add discount codes (percent or flat) with validity window, minimum amount and a maximum number of uses. Usage count must stay correct even under parallel use - another concurrency problem.
API Endpoints	`POST /api/v1/bookings/:id/apply-promo` `POST /api/v1/admin/promos`
Validation & Rules	Code case-insensitive; expired/exhausted code -> 409; discount never makes total negative.
Database	**promo_codes** - id UUID PK - code VARCHAR(20) NOT NULL UNIQUE - discount_type VARCHAR(10) NOT NULL CHECK IN (PERCENT, FLAT) - discount_value INT NOT NULL - min_amount_paise INT NOT NULL DEFAULT 0 - max_uses INT NULL - used_count INT NOT NULL DEFAULT 0 - valid_from / valid_until TIMESTAMPTZ NOT NULL - is_active BOOLEAN NOT NULL DEFAULT true
Done When	**100 parallel uses of a 10-use code give exactly 10 successes.**

10.3 Full-text search

Description	Use PostgreSQL full-text search (tsvector + GIN index) for movie search with ranking and typo tolerance (pg_trgm).
API Endpoints	`GET /api/v1/movies/search?q=`
Validation & Rules	Minimum 2 characters; limit results.
Database	`Add to movies: search_vector TSVECTOR + GIN index`
Done When	**Searching a partial/misspelled title still finds the movie.**

10.4 Audit logs & outbox pattern

Description	Record important admin/organizer actions in an audit log. Implement the outbox pattern so events (booking confirmed) are written in the same transaction as the data and published reliably afterwards.

Validation & Rules	Audit logs are append-only.
Database	**audit_logs** - id BIGSERIAL PK - actor_id UUID NULL FK users(id) - action VARCHAR(50) NOT NULL - entity_type VARCHAR(30) NOT NULL - entity_id VARCHAR(40) NOT NULL - metadata JSONB NULL - ip_address INET NULL - created_at TIMESTAMPTZ NOT NULL DEFAULT now() **outbox_events** - id BIGSERIAL PK - aggregate_type VARCHAR(30) NOT NULL - aggregate_id VARCHAR(40) NOT NULL - event_type VARCHAR(40) NOT NULL - payload JSONB NOT NULL - published_at TIMESTAMPTZ NULL - created_at TIMESTAMPTZ NOT NULL DEFAULT now()
Done When	**No event is lost even if the app crashes right after commit.**

10.5 Architecture write-up

Description	Write README with architecture diagram, ER diagram, booking sequence diagram (hold > pay > webhook > confirm), ADRs, load test results and a 'what I would do at 100x traffic' section.
Done When	**A stranger can understand and run the project from README alone.**

Edge Cases to Test

These are the scenarios that separate a working demo from a reliable backend. Write an automated test for each one.

- Two users try to book the same seat at the same time.
- Two users book overlapping seat sets (for example A1+A2 and A2+A3) - check for deadlocks.
- Payment arrives after the hold has expired.
- The same webhook is delivered several times, or out of order.
- Payment succeeds but the server crashes before the webhook is processed.
- User closes the browser in the middle of payment.
- A show is cancelled while confirmed bookings exist.
- Two parallel requests arrive with the same Idempotency-Key.
- Redis goes down - does the system degrade gracefully?
- The worker crashes in the middle of an email job.
- A refresh token is used more than once.
- An organizer tries to edit a resource that belongs to someone else.

Suggested Timeline

Assumes about 1.5 to 2 hours per day, roughly 12 weeks in total. Adjust it around your freelance work.

- Week 1: Phase 0 (setup and foundation)
- Weeks 2-3: Phase 1 (authentication, OAuth)
- Weeks 3-4: Phase 2 and Phase 3 (catalog, seat inventory)
- Weeks 5-6: Phase 4 (booking engine)
- Week 7: Phase 5 (payments and refunds)
- Week 8: Phase 6 (queues and notifications)
- Week 9: Phase 7 (caching, security, performance)
- Week 10: Phase 8 (testing gaps and coverage)
- Week 11: Phase 9 (Docker, CI/CD, deployment)
- Week 12: Phase 10 (bonus features, README, diagrams, load test report)